Tutorial 10

Definition 1.1

A flow network is a directed graph $G = (V, E)$ where each edge has a nonnegative capacity. We formalize this by saying there is a map $c : E \to \mathbb{R}_{\geq 0}$.

Furthermore, G is such that $(u, v) \in E \Rightarrow (v, u) \notin E$

Definition 1.2

Let two different vertices $s, t \in V$ be given; call them the source and sink, respectively. A flow is a map $f : E \to \mathbb{R}_{\geq 0}$, satisfying:

- Edge rule: for every edge e, $0 \leq f(e) \leq c(e)$ aka the flow on every edge is nonnegative and no larger than the capacity of the edge.
- Vertex rule: For every vertex v other than s, t

$$\sum_{x \in N_{\text{in}}(v)} f(x, v) = \sum_{x \in N_{\text{out}}(v)} f(v, x)$$

that is the sum of the flows on incoming edges is equal to the sum of the flows on outgoing edges.

Definition 1.3

We define the value of flow as:

$$|f| = \sum_{x \in N_{\text{out}}(s)} f(s, x) - \sum_{x \in N_{\text{in}}(s)} f(x, s)$$

Definition 1.4

An $s - t$ cut is a partition $V = S \cup T$ where S and T are disjoint and $s \in S$, $t \in T$, and the cost of an $s - t$ cut is

$$c(S, T) = \sum_{x \in S,\, y \in T,\, (x,y) \in E} c(x, y)$$

Exercise 1

Prove that for any flow f and any $s - t$ cut (S, T) of G, we have $|f| \leq c(S, T)$.

In particular, we want to show that the value of the max flow is at most the value of the min cut.

Exercise 2

Prove that if we can find f and (S, T) such that $|f| = c(S, T)$, then f is a max flow and (S, T) is a min $s - t$ cut.

Definition 1.5

We define a residual graph G_f where the residual capacities $c_f : V \times V \to \mathbb{R}_{\geq 0}$ are defined as follows:

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E \\ f(v, u) & \text{if } (v, u) \in E \\ 0 & \text{otherwise} \end{cases}$$

Basically, what this does is that, if there is any flow through the edge, you remove the flow from the capacity and add an edge in the opposite direction with the value of the flow.

Exercise 3

Show that if t is not reachable from s is G_f, then f is a maximum flow.

Exercise 4

Show that if G_f has a path from s to t, we can modify f to f' such that $|f| < |f'|$

Hint: Consider if there is a simple path P in G_f such that $s = x_0 \to x_1 \to \cdots \to x_k = t$, and let $F = \min c_f(x_i, x_{i+1})$. We define a new flow f' where:

$$f'(u,v) = \begin{cases} f(u,v) + F & \text{if } (u,v) \in P \\ f(u,v) - F & \text{if } (v,u) \in P \\ f(u,v) & \text{otherwise} \end{cases}$$

Exercise 5

Can you use the above exercises to create an algorithm that finds the maximum flow?

Also, can you use this to conclude that min-cut and max-flow are equivalent?

Template Max-Flow Algorithm

Show that this algorithm runs in $O(|f|m)$ time:

1: $f \leftarrow$ all zeroes flow
2: $G_f \leftarrow G$
3: **while** t is reachable from s in G_f (check using, DFS or BFS etc): **do**
4: $P \leftarrow$ path in G_f from s to t
5: $F \leftarrow$ min capacity on P
6: $f' \leftarrow f$ as defined above
7: Update G_f to correspond to new flow
8: **end while**
9: **return** f

Note we call the chosen path P the augmenting path and the update f' as the augmentation.

Ford-Fulkerson Algorithm

Say we have a black box algorithm that given a graph G finds a path between vertices s and t such that maximizes $\min_{e \in P} c_f(e)$ among all $s - t$ paths P in in $O(m + n)$ time.[1]

The Ford-Fulkerson Algorithm uses the above blackbox to choose the augmenting path. Show that for graph with integer capacities, the Ford-Fulkerson algorithm runs in $O((m + n)m \log |f|)$ time.

Hint: You might want to first show that if we let f' be the max flow in G_f. Then after one iteration of Ford-Fulkerson on G_f, the max flow on G_f becomes $\leq |f'| \left(1 - \frac{1}{m}\right)$

Hint: Does our choice of augmenting path P give us a way to cut G_f (consider what the minimum capacity edge is..)? What can be said about the value of flow? Perhaps using the min-cut maxflow duality...

[1] This is indeed possible by a clever mix of median finding + DFS

Edmonds-Karp algorithm

In this algorithm, we choose the augmenting path between s and t using BFS, thus picking a path that minimizes the number of edges.

Note, this runs in $O(n+m) = O(m)$ time as we only check the connected component of s and in connected graphs, $m \geq n-1$.

Show that this Edmonds-Karp algorithm runs in $O(nm^2)$.

Hint: You might first want to show for any $u, v \in V$ such that (u, v) is an edge in G_f at some point, then the number of times that the edge (u, v) can disappear from G_f is at most $\frac{n}{2}$.

Hint: Let G_f^i be the residual network after the i-th iteration of the algorithm; $G_f^0 = G$. For a vertex v, let $d_i(v)$ be the (unweighted) distance from s to v in G_f^i. Now show that for all $i \geq 1$, and all $v \in V$, $d_{i-1}(v) \leq d_i(v)$.

Hint: For the above, induct on the distance from s!

For all the below, assume we can find the min-cut or max-flow in $O(nm^2)$ time.

Verification

Given G and f, design an $O(n + m)$ time algorithm to determine if the flow f is a maximum flow in G.

Flow Update I

Suppose that the capacity of a single edge $e \in E$ is increased by 1. Given G, its maximum flow f before the update and the edge to be updated e, design an $O(n+m)$ time algorithm to find a new maximum flow f'.

Flow Update II

Suppose that the capacity of a single edge $e \in E$ is decreased by 1. Given G, its maximum flow f before the update and the edge to update e, design an $O(m+n)$ time algorithm to find a new maximum flow f'.

Loops

Our graph G had the condition $(u, v) \in E \Rightarrow (v, u) \notin E$. Can you give an algorithm to find the max flow in $O(m^2(m + n))$ time?

Max Flow with Parallel Edges

Suppose G is allowed to have multiple directed edges from u to v, possibly with different capacities. Can you give an algorithm to find a maximum flow in $O(nm^2)$ time?

Multiple Sources and Sinks

Suppose the graph has sources $s_1, \ldots, s_k$ and sinks $t_1, \ldots, t_r$. Flow may originate at any source and terminate at any sink. Reduce this problem to a standard single-source, single-sink max-flow problem.

Max Flow with Vertex Capacities

Suppose every vertex v (except the source s and sink t) has a capacity $c(v)$, limiting the total amount of flow that can pass through it.

Find the max flow in $O((m+n)^2 n)$ time.

Minimum Number of Edges in a Cut

Find an s-t cut that contains the minimum possible number of edges, regardless of their capacities. Can this problem be reduced to an ordinary min-cut problem? Give a reduction and prove its correctness.

A Cut with a Prescribed Vertex

Given a vertex v, find a minimum s-t cut such that v belongs to the same side as s. Give an algorithm that runs in $O(m^2 n)$ time.